

Medical Cost - Zadanie projektowe

Opis problemu

Zadaniem jest stworzenie, wytrenowanie oraz ocena modeli regresyjnych przewidujących roczne koszty leczenia ubezpieczonego na podstawie jego cech demograficznych i zdrowotnych. Projekt wykorzystuje zestaw danych “Medical Cost Personal Dataset”, zawierający informacje o klientach amerykańskiego ubezpieczyciela zdrowotnego.

Zbiór danych

- Źródło: [Medical Cost Personal Dataset](#) (Kaggle / publiczne mirrory)
- Plik: `insurance.csv`
- Rozmiar: **1338** rekordów, **7** kolumn

Struktura danych

Kolumna	Typ	Opis
age	int	wiek ubezpieczonego
sex	str	pleć (female , male)
bmi	float	wskaźnik masy ciała (Body Mass Index)
children	int 0-5	liczba dzieci na utrzymaniu
smoker	str	czy osoba pali (yes , no)
region	str	region (northeast , northwest , southeast , southwest)
charges	float	ZMIENNA CELU - roczne koszty leczenia (USD)

Zbiór nie zawiera braków danych. Cechy dzielą się na **kategoryczne** (**sex**, **smoker**, **region**) oraz **numeryczne** (**age**, **bmi**, **children**).

Wymagania formalne

Forma rozwiązania

Rozwiązanie składa się z **dwóch plików**:

1. **solution.py** - moduł Python z implementacją wszystkich wymaganych funkcji
2. **wnioski.md** - dokument Markdown z interpretacją wyników i wnioskami

Wymagania dla solution.py

1. Użyj dostarczonego szablonu **solution_template.py**
2. **Nie zmieniaj** nazw funkcji, parametrów ani typów zwracanych
3. Zaimplementuj **wszystkie** funkcje oznaczone **# TODO**
4. Możesz dodawać własne funkcje pomocnicze (nazwy zaczynające się od **_**)
5. Kod musi być uruchamialny bez błędów
6. Funkcja **run_full_analysis()** musi zwracać kompletny słownik wyników

Wymagania dla wnioski.md

Dokument powinien zawierać następujące sekcje (użyj pliku **wnioski_template.md**):

1. **Analiza eksploracyjna** (max 200 słów)
 - Kluczowe obserwacje z analizy danych
 - Opis rozkładu zmiennej celu
 - Najsilniejsze zależności (numeryczne i grupowe)
2. **Porównanie modeli** (max 200 słów)
 - Który model osiągnął najlepsze wyniki i dlaczego?
 - Różnice między modelami prostymi a zespołowymi
 - Analiza overfittingu (porównanie train vs test)
3. **Interpretacja cech** (max 200 słów)
 - Top 3 najważniejsze cechy według RandomForest
 - Top 3 najważniejsze cechy według LinearRegression
 - Czy modele wskazują te same cechy? Wyjaśnienie różnic
4. **Analiza reszt** (max 100 słów)
 - Ocena wykresu reszt - czy błędy są losowe?
 - Czy występuje systematyczne niedoszacowanie/przeszacowanie?
5. **Wnioski i rekomendacje** (max 150 słów)

- Podsumowanie projektu
- Propozycje ulepszeń

Wymagane załączniki (obrazki):

- Wykres korelacji (heatmapa)
 - Wykres `y_true` vs `y_pred` dla najlepszego modelu
 - Wykres reszt
 - Wykres ważności cech (RandomForest)
-

Specyfikacja funkcji

Część 1: Wczytanie i eksploracja danych

`load_data(filepath: str) -> pd.DataFrame`

Wczytuje surowe dane z pliku CSV.

`get_basic_info(df: pd.DataFrame) -> Dict[str, Any]`

Zwraca słownik:

- 'shape': Tuple[int, int]
- 'dtypes': pd.Series
- 'missing_values': pd.Series
- 'duplicates_count': int

`get_target_statistics(df: pd.DataFrame) -> Dict[str, float]`

Zwraca słownik ze statystykami zmiennej `charges`:

- 'mean', 'median', 'std', 'min', 'max', 'q1', 'q3'

```
get_feature_info(df: pd.DataFrame, feature: str) -> Dict[str, Any]
```

Zwraca słownik:

- 'dtype': str
 - 'unique_values': np.ndarray
 - 'unique_count': int
 - 'describe': pd.Series
-

Część 2: Analiza eksploracyjna

```
get_correlation_matrix(df: pd.DataFrame) -> pd.DataFrame
```

Zwraca macierz korelacji dla cech numerycznych (age, bmi, children) oraz zmiennej celu (charges).

```
get_group_statistics(df: pd.DataFrame) -> Dict[str, pd.Series]
```

Zwraca słownik ze średnią charges w podziale na grupy:

- 'by_smoker': średnia charges wg palenia
- 'by_region': średnia charges wg regionu
- 'by_sex': średnia charges wg płci

```
check_distribution(df: pd.DataFrame) -> Dict[str, float]
```

Zwraca słownik:

- 'skewness', 'kurtosis' - dla charges
 - 'log_skewness', 'log_kurtosis' - dla log(1+charges)
-

Część 3: Przygotowanie danych

```
prepare_features(df: pd.DataFrame) -> Tuple[pd.DataFrame, pd.Series]
```

Zwraca (X, y) - cechy oraz zmienną celu charges.

`get_column_lists()` -> Dict[str, List[str]]

Zwraca słownik:

- 'categorical': ['sex', 'smoker', 'region']
- 'numerical': ['age', 'bmi', 'children']

`create_preprocessor()` -> ColumnTransformer

Zwraca preprocessor z OneHotEncoder i StandardScaler.

`split_data(X, y, test_size=0.2, random_state=42)` -> Tuple[...]

Zwraca (X_train, X_test, y_train, y_test).

Część 4: Budowa modeli

`get_models()` -> Dict[str, Any]

Zwraca słownik z modelami:

- 'LinearRegression': LinearRegression()
- 'Ridge': Ridge(alpha=1.0)
- 'Lasso': Lasso(alpha=0.1)
- 'DecisionTree': DecisionTreeRegressor(max_depth=5, random_state=42)
- 'RandomForest': RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42)
- 'GradientBoosting': GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)

`train_model(model, X_train, y_train)` -> Any

Trenuje i zwraca model.

Część 5: Ocena modeli

`calculate_metrics(y_true, y_pred)` -> Dict[str, float]

Zwraca: 'MAE', 'MSE', 'RMSE', 'R2'

```
evaluate_all_models(models, X_train, X_test, y_train, y_test) -> pd.DataFrame
```

Zwraca DataFrame z kolumnami:

Model, MAE_train, MSE_train, RMSE_train, R2_train, MAE_test, MSE_test, RMSE_test, R2_test

```
get_best_model_name(results_df, metric='R2_test') -> str
```

Zwraca nazwę najlepszego modelu.

```
get_predictions(model, X) -> np.ndarray
```

Zwraca predykcje.

```
calculate_residuals(y_true, y_pred) -> np.ndarray
```

Zwraca reszty.

Część 6: Interpretacja modelu

```
get_feature_importance(model, feature_names) -> pd.DataFrame
```

Zwraca DataFrame (feature, importance) posortowany malejąco.

```
get_linear_coefficients(model, feature_names) -> pd.DataFrame
```

Zwraca DataFrame (feature, coefficient) posortowany po |coefficient|.

```
get_top_features(importance_df, n=3) -> List[str]
```

Zwraca listę n najważniejszych cech.

Część 7: Wizualizacje

Wszystkie funkcje wizualizacyjne zwracają `plt.Figure`:

- `plot_target_distribution(df)` - histogram charges
- `plot_correlation_heatmap(corr_matrix)` - heatmapa
- `plot_feature_vs_target(df, feature)` - scatter/box plot
- `plot_group_statistics(group_stats)` - 3 subplots
- `plot_predictions_vs_actual(y_true, y_pred)` - scatter z linią $y=x$
- `plot_residuals(y_pred, residuals)` - wykres reszt
- `plot_feature_importance(importance_df, top_n=10)` - bar plot
- `plot_coefficients(coef_df, top_n=10)` - bar plot

`generate_all_figures(df, results, output_dir="figures") -> Dict[str, str]`

Generuje i zapisuje wszystkie wykresy do wskazanego folderu. Zwraca słownik z nazwami wykresów i ścieżkami do plików.

Funkcja główna

`run_full_analysis(filepath: str) -> Dict[str, Any]`

Uruchamia pełną analizę i zwraca słownik ze wszystkimi wynikami.

System oceniania

Etap	Opis	Punkty
1. Wczytanie i eksploracja	Funkcje <code>load_data</code> , <code>get_basic_info</code> , <code>get_target_statistics</code> , <code>get_feature_info</code>	10 pkt
2. Analiza eksploracyjna	Funkcje <code>get_correlation_matrix</code> , <code>get_group_statistics</code> , <code>check_distribution</code>	10 pkt
3. Przygotowanie danych	Funkcje <code>prepare_features</code> , <code>get_column_lists</code> , <code>create_preprocessor</code> , <code>split_data</code>	10 pkt
4. Budowa modeli	Funkcje <code>get_models</code> , <code>train_model</code>	10 pkt
5. Ocena modeli	Funkcje <code>calculate_metrics</code> , <code>evaluate_all_models</code> , <code>get_best_model_name</code> , etc.	15 pkt
6. Interpretacja	Funkcje <code>get_feature_importance</code> , <code>get_linear_coefficients</code> , <code>get_top_features</code>	10 pkt
7. Wizualizacje	Wszystkie funkcje <code>plot_*</code>	15 pkt
8. Integracja	Funkcja <code>run_full_analysis</code>	5 pkt
9. Wnioski.md	Dokument z interpretacją i obrazkami	15 pkt

Łącznie: 100 punktów

Materiały pomocnicze

- [Scikit-learn dokumentacja](#)
- [Pandas dokumentacja](#)
- [Matplotlib dokumentacja](#)
- [Seaborn dokumentacja](#)